COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Jul 28, 2026	**DATE SUBMITTED:**	Aug 25, 2026
NAME:	Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4 **Stacks**

1. PROCEDURE OUTPUT

ILO A

```cpp
#include <iostream>
#include <stack>

int main(){
    std::stack<int> newStack; // created an object

    //testing the empty() function to check if the stask is empty
    std::cout<<"is the stack empty? " << newStack.empty()<<std::endl;

    //insert 3 elements:
    newStack.push(1);
    newStack.push(2);
    newStack.push(3);

    std::cout<<"is the stack empty? " << newStack.empty()<<std::endl;

    newStack.push(3);
    std::cout<<"Top of stack: "<<newStack.top()<<std::endl;

    newStack.push(2);
    std::cout<<"Top of stack: "<<newStack.top()<<std::endl;

    newStack.push(1);
    std::cout<<"Top of stack: "<<newStack.top()<<std::endl;

    std::cout<<"is the stack empty? " << newStack.empty()<<std::endl;
    newStack.push(6);
    newStack.push(7);
    newStack.push(8);

    std::cout<<"Size of stack: "<<newStack.size()<<std::endl;

    return 0;
}

```

DATA ANALYSIS:

The stack uses LIFO in managing data; this means that data input recently is at the top which you can do with the function push(), pushing the data down. It also accesses the top first using the function top(). When you need to remove data, you can only remove what the stack can only access, which is the top, you can do this with the function pop(). If you need to know if the stack is empty, you can use the function empty() which returns a bool value of 1 meaning that the stack is empty, otherwise if not. The function full(), acts the same as empty(). It checks if the stack is full and returns the bool true if it is true. There is also another function that returns the size of the stack using size().

OBSERVATIONS:

The concept of stack is easy to understand. It follows the LIFO rule “Last In, First Out”. When comparing this to Linked Lists, the process is much easier to digest. Since it follows the Last data In, it is the First Out or first accessed by the stack.

ILO B

```cpp
#include

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;

//protoyping
bool isEmpty();
bool isFull();
void push();
void pop();
void Top();
void displayALL();

int main(){
    int choice;

    //asked the user for the maximum elements to be inserted
    std::cout<<"enter the number of maximum of elements for the new stack (max:100): ";
    std::cin>>elements;

    while(true){
        std::cout <<"\nstack Operations: "<
        std::cout << "1. PUSH\n"
                  << "2. POP\n"
                  << "3. TOP\n"
                  << "4. isEmpty\n"
                  << "5. isFull\n"
                  <<"6. Display Stack\n"
                  << "7. Exit\n"
                  << "Enter Choice: "<
        std::cin>>choice;
```

```
        switch (choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<
            break;
            case 5: std::cout<
            break;
            case 6: displayALL();
            break;
            case 7: return false;
            break;
            default: std::cout<<"invalid choice."<
            break;
        }
    }

    return 0;
}
bool isEmpty(){
    //top is -1
    if(top == -1){
        std::cout<<"Stack Underflow."<
        return true;
    }
    return false;
}

bool isFull(){
    //not
    if(top >= elements){
        std::cout<<"Stack Overflow"<
        return true;
    }
    return false;
}
```

```cpp
void push(){
    //check if the stack is full, if yes return an error
    if (isFull()) return;
    //ask the user for a value to be pushed into the stack
    std::cout << "What number you want to add to the stack?: ";
    std::cin >> newData;
    stack[++top] = newData;
}
void pop(){
    //check if stack is empty
    if(isEmpty()) return;

    //display the element to popped
    std::cout<<"POPPED element: 0"<<stack[top]<<std::endl;

    //decrement the top
    top--;
}
void Top(){
    if(isEmpty()) return;

    std::cout<<"The TOP element: "<<stack[top]<<std::endl;
}
void displayALL(){
    if(isEmpty()) return;
    std::cout<<"Printing from TOP to BOTTOM: "<<std::endl;
    for(int i = 0; i < elements; i++){
        std::cout<<stack[i]<<" ";
    }
}
```

CODE ANALYSIS:

For this activity, we learned how to create the templates for the stack operations and really try to understand the process behind these functions. We created the following functions: push(), pop(), Top(), isEmpty(), isFull(), isEmpty(), and displayALL(). Starting with isEmpty(), to know if a stack is empty we need to understand how a stack functions. A stack functions with the rule LIFO. This means that the data they can access is the top. So if the top is equal to -1, this means there is no data stored and that the stack is empty. Next, isFull() can tell if a stack is full with the condition that the top is greater than the number of elements in the stack. If the stack is full the function returns true. Next is push(), this function allows the user to input or to “push” the data into the bottom of the stack. Then the top of the array is incremented to adjust the current top which is a placeholder for data. If you use top++, it returns the top element as 0. Maybe because it’s not really pointing to the recent inputted data? Next, the pop() which asks if the stack is empty as a precaution then it prints the element to be popped. It then reassigns the top to the previous top. The top() function just looks at the stack[top] is assigned to. Then for displayALL(), there is a better way of doing it, but what I did was I printed it as an array using a for loop.

OBSERVATIONS:

I noticed that I misinterpreted the way I could present the displayALL() function as I printed it like an array. When I should have printed it like a stack just the way it was intended, “LIFO”. I did think about displaying it as it.

CODE OUTPUT:

```
enter the number of maximum of elements for the new stack (max:100): 2

Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display Stack
7. Exit
Enter Choice:
1
What number you want to add to the stack?: 24
```

```
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display Stack
7. Exit
Enter Choice:
1
What number you want to add to the stack?: 67
```

```
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display Stack
7. Exit
Enter Choice:
4
0
```

```
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display Stack
7. Exit
Enter Choice:
5
0
```

```
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display Stack
7. Exit
Enter Choice:
6
Printing from TOP to BOTTOM:
67 0
```

```
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display Stack
7. Exit
Enter Choice:
2
POPPED element: 00
```

```
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display Stack
7. Exit
Enter Choice:
7
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Stacks Using Arrays

```
111  void checkBalanceA(){
112      std::string character;
113      std::cout<<"Enter Expression: ";
114      std::cin>>character;
115  
116      //read each character
117      for(int i = 0; i < character.length(); i++){
118          //opening symbols
119          if(character[i] == '(' || character[i] == '{' || character[i] == '['){
120              pushA(character[i]);
121          }
122  
123          //closing symbols
124          if(character[i] == ')' || character[i] == '}' || character[i] == ']'){
125              //if the stack is empty error catch
126              if(isEmptyA()){
127                  std::cout<<"Error, unbalanced expression: Stack is empty."<<std::endl;
128                  return;
129              }
130  
131              //check if the symbol popped corresponds to the closing symbol
132              if((stack[top] == '(' && character[i] != ')') ||
133                 (stack[top] == '{' && character[i] != '}') ||
134                 (stack[top] == '[' && character[i] != ']')){
135                  std::cout<<"Error, unbalanced expression: not corresponding to opening symbol."<<std::endl;
136                  return;
137              }
138  
139              //pop the stack
140              popA();
141          }
142      }
143  
144      //check if the stack is not empty
145      if(top != -1){
146          std::cout<<"Error, unbalanced expression: Stack is not empty."<<std::endl;
147      }
148      else{
149          std::cout<<"Valid: Y"<<std::endl;
150      }
151  }
```

Stacks Using Linked Lists

```
template <typename T>
void checkBalanceLL(){
    std::string character;
    std::cout<<"Enter Expression: ";
    std::cin>>character;

    //read each character
    for(int i = 0; i < character.length(); i++){
        //opening symbols
        if(character[i] == '(' || character[i] == '{' || character[i] == '['){
            push(character[i]);
        }

        //closing symbols
        if(character[i] == ')' || character[i] == '}' || character[i] == ']'){
            //if the stack is empty error catch
            if(isEmpty<T>()){
                std::cout<<"Error, unbalanced expression: Stack is empty."<<std::endl;
                return;
            }

            //pop the stack
            char temp = pop<T>();

            //check if the symbol popped corresponds to the closing symbol
            if((temp == '(' && character[i] != ')') ||
               (temp == '{' && character[i] != '}') ||
               (temp == '[' && character[i] != ']')){
                std::cout<<"Error, unbalanced expression: not corresponding to opening symbol."<<std::endl;
                return;
            }
        }
    }

    //check if the stack is not empty
    if(!isEmpty<T>()){
        std::cout<<"Error, unbalanced expression: Stack is not empty."<<std::endl;
    }
    else{
        std::cout<<"Valid: Y"<<std::endl;
    }
}
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	**Using Arrays:** Checking for Balanced Symbols.. Test Using: A) Stack using Arrays B) Stack using Linked Lists C) EXIT Choice: A Enter Expression: (A+B)+(C-D) Popping: (Popping: (Valid: Y	The operation pushes the opening delimiter (into the stack while ignoring characters that are not opening or closing delimiters. When the closing delimiter) is encountered, the (is checked and popped from the stack. The same process happens again with the second pair of parentheses. Since the stack becomes empty after all the delimiters are checked, the expression is considered valid.

		Using Linked Lists: Checking for Balanced Symbols. Test Using: A) Stack using Arrays B) Stack using Linked Lists C) EXIT Choice: B Enter Expression: (A+B)+(C-D) Valid: Y	
((A+B)+(C-D)	N	**Using Arrays:** Checking for Balanced Symbols.. Test Using: A) Stack using Arrays B) Stack using Linked Lists C) EXIT Choice: A Enter Expression: ((A+B)+(C-D) Popping: (Popping: (Error, unbalanced expression: Stack is not empty. **Using Linked Lists:** Checking for Balanced Symbols.. Test Using: A) Stack using Arrays B) Stack using Linked Lists C) EXIT Choice: B Enter Expression: ((A+B)+(C-D) Error, unbalanced expression: Stack is not empty.	For this expression, the opening delimiter (is pushed onto the stack twice. The other characters that are not opening or closing delimiters are ignored. When the operation comes across a closing delimiter that matches the one on top, that opening delimiter is popped. This leaves one opening delimiter at the top. Then another opening delimiter gets pushed into the stack and a closing delimiter is evaluated, causing the opening delimiter to be popped. This leaves one opening delimiter, making the expression unbalanced because the stack is not empty.
((A+B)+[C-D])	Y	**Using Arrays:** Checking for Balanced Symbols.. Test Using: A) Stack using Arrays B) Stack using Linked Lists C) EXIT Choice: A Enter Expression: ((A+B)+[C-D]) Popping: (Popping: [Popping: (Valid: Y	The expression contains three matching pairs of opening and closing delimiters.As shown in the trace, each stored opening delimiter is popped sequentially during evaluation, resulting in an empty stack upon completion.

		Using Linked Lists: Checking for Balanced Symbols.. Test Using: A) Stack using Arrays B) Stack using Linked Lists C) EXIT Choice: B Enter Expression: ((A+B)+[C-D]) Valid: Y	
((A+B]+[C-D]}	N	**Using Arrays:** Checking for Balanced Symbols.. Test Using: A) Stack using Arrays B) Stack using Linked Lists C) EXIT Choice: A Enter Expression: ((A+B]+[C-D]} Error, unbalanced expression: not corresponding to opening symbol. **Using Linked Lists:** Checking for Balanced Symbols.. Test Using: A) Stack using Arrays B) Stack using Linked Lists C) EXIT Choice: B Enter Expression: ((A+B]+[C-D]} Error, unbalanced expression: not corresponding to opening symbol.	In this expression, the stack ends up holding two opening parentheses since (appears twice before any closing symbol shows up. The letters and the + sign are ignored because they are not opening or closing delimiters. When the operation reaches the closing delimiter], it checks the top of the stack, which is (, not [. Since the closing symbol does not match the opening symbol on top, the program stops immediately and shows an error instead of continuing through the rest of the expression. This happens the same way in both the array-based and linked list-based stacks because they follow the same checking process, they just store the symbols differently. This shows that an expression is not automatically balanced just because it has matching numbers of opening and closing symbols, what matters is whether each closing symbol corresponds to the correct opening symbol.

Conclusion

In this activity, I learned how a stack works as a linear data structure that follows the LIFO rule, where the last data pushed is the first one to be accessed and removed, and I was able to apply this by implementing a stack using the C++ STL as well as building it manually using arrays and linked lists, which helped me understand what actually happens behind built-in functions like push(), pop(), top(), and isEmpty(). Building the stack using arrays was straightforward since I only had to keep track of the top index and adjust it whenever I pushed or popped data, while using linked lists took more steps since I had to manage the head pointer and create new nodes for every push operation, though both still follow the same LIFO order, with arrays being easier to set up and linked lists being better suited for cases where the number of elements is not fixed. For the supplementary activity, I applied the stack to check if an expression has balanced symbols, which showed me a practical use of a stack since the program pushes opening delimiters and pops them when a matching closing delimiter appears, and testing different expressions helped me see that an expression is not automatically valid just because it has an equal number of opening and closing symbols, what matters is whether each closing symbol corresponds to the correct opening symbol and whether the stack ends up empty after the whole expression is read. Overall, I think I did well in this activity since I was able to complete both the array-based and linked list-based implementations and apply them to the balanced symbols problem, though my area for improvement is on planning my implementation first before coding, since I realized this when I misinterpreted how displayALL() should present the stack, and I also want to get more comfortable with linked list-based implementations since I still find them slightly more complex compared to arrays.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, summarization, code assistance, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used AI to adjust my grammar, summarize, helped me with errors, and explained concepts I'm unfamiliar with.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines